
Does the simulator behave like a real machine?

The simulator has one heartbeat: work makes heat, heat slows the chip, and a slower chip does less work. Before we trust it, we check that heartbeat against real GPUs in three stages.

1

MEASURE EACH STEP

2

CHECK THE SHAPE ON HEALTHY DAYS

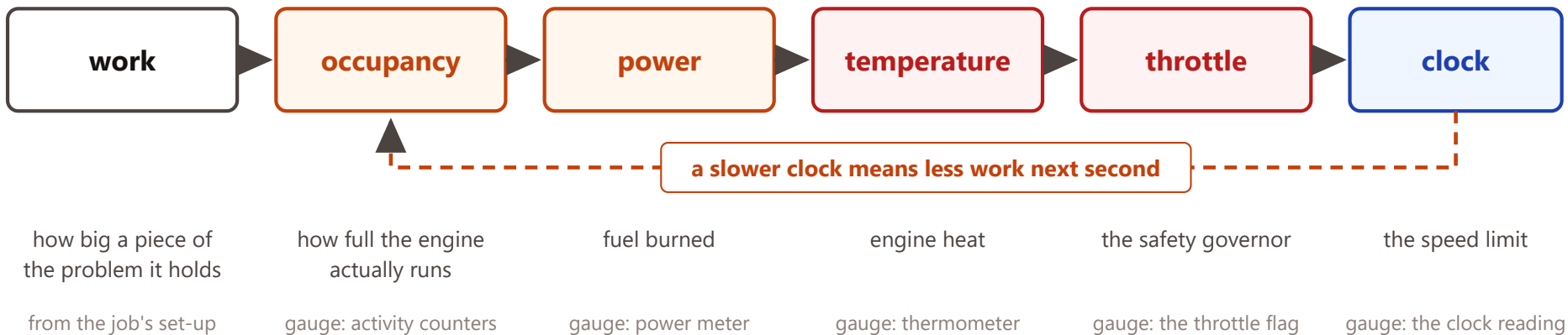
3

REHEARSE THE FAULTS, SAFELY

THE HEARTBEAT

Six things in a circle, and every arrow can be measured

Think of a GPU as an engine. The loop below is the whole physics of the simulator, and each box has a real gauge on a real GPU.



Every real GPU reports all five gauges about once a second. Calibration is simply asking the real machine the same five questions the simulator answers.

THE SUBTLE ONE

"Busy" is not the same as "working"

A GPU has a busy light that is on almost all the time, even when it is only waiting for the slowest team member. The simulator ignores the busy light and uses **active occupancy**: how much real work came off the line each second.

The factory-line picture

A machine can be switched on all day and still produce nothing while it waits for parts. The busy light says "on". Active occupancy counts the parts that actually came off the line. Only the second one tells you a sick GPU is holding up 127 others.

How active occupancy is put together

Time spent computing × **how full the engine gets for a piece that size**, plus **time spent waiting** × almost nothing. The "how full" part depends only on the size of the piece, and is measured once in the lab. The two time shares come from the job's own stopwatch.

Three instruments, three jobs

1

The fleet dashboard

Every GPU, once a second, always on. Reads the activity counters, the busy light, power, temperature, throttle and clock. This is what the detector runs on. (DCGM)

2

The lab microscope

Looks inside one program on one GPU, once. Tells us how full the engine gets for each piece size. Too slow to use in production. (Nsight Compute)

3

The job's own stopwatch

Marks when each rank is computing, exchanging data or waiting. It is the only way to see the waiting. (Nsight Systems)

One afternoon on one machine fits every constant

Each arrow in the loop gets one simple experiment. Because each experiment touches only one arrow, a bad result points at exactly one equation.

1

The size sweep

Run the solver on pieces from tiny to huge. Watch how full the engine gets. This draws the curve the simulator uses for occupancy.

2

The speed sweep

Pin the clock at five speeds and read the power meter. Power should rise a little faster than the square of the speed.

3

The heat step

Switch on full load, watch the thermometer climb for five minutes, switch off, watch it fall. That gives how hot, and how fast.

4

Read the label

The throttle temperature is printed on the device. Old records show at what temperature it lets go again.

5

Watch the staircase

When a GPU slows down it steps, never slides. The step size is on the device too, and every recorded slowdown must land on those steps.

Nothing here needs a failure to have happened. Fourteen numbers in the simulator, five measurements, one afternoon.

Does an ordinary healthy day look the way the loop says it should?

Before any fault is involved, real data has to agree with the simulator about the shape of things. These are yes-or-no checks on normal production days.

- ✓ **Power follows work, not the busy light**
When real work drops, power drops, even though the busy light stays on.
- ✓ **Heat rises and falls the same simple way every time**
Every job start on every GPU should fit one warm-up time, about 20 seconds.
- ✓ **A whole rack warms up together**
Cooling is shared per rack, so 32 GPUs should move as one.
- ✓ **The throttle is a staircase, and reacts one beat late**
Clocks step down in fixed notches, one second after the temperature crosses the line.
- ✓ **Waiting really does look like waiting**
A rank parked at the barrier shows the busy light on and almost no work coming off the line.
- ✓ **The team moves at the pace of its slowest member**
Every rank's computing plus waiting adds up to the same iteration time.

A failed check is not a number to tweak. It is a piece of physics the simulator is missing, and it tells us which one.

Stage each fault safely, then compare the story

Every scenario in the simulator has a harmless real rehearsal on a spare node or rack. We compare the story, not the decimals: what moves first, what stays flat, and whether the diagnosis names the right suspect.

Simulated fault	Safe real rehearsal
Cooling fails on one rack	Turn that rack's air-conditioning up a few degrees in a maintenance window; 32 GPUs should warm together.
One GPU goes bad	Cap one GPU's clock at 90 % while a job runs. Everyone else should end up waiting for it.
A rack's network link degrades	Switch off 7 of the 8 cables from one rack's switch. Only the cross-rack traffic should slow.
A leftover process steals time	Pin a stray program to one host. The victim should stumble on and off, never continuously.
The job simply writes more output	Change the job's checkpoint setting. Everything slows evenly and no hardware gauge moves: the fault that is not a fault.

What passing looks like

Same order of events, same gauges moving, same gauges flat, and the diagnosis pointing at the same rack or GPU as the ticket.

Afterwards: real history

Every past incident with a known cause is replayed through the same detector and scored the same way. Where it fails, we have found the next piece of physics to add.

The loop stays. The numbers get re-measured.

A different workload or a different failure does not need a different simulator. It needs the same three stages run again, on the arrows that changed.

A different kind of job

An AI training run instead of fluid dynamics uses a different part of the engine and passes far more data at the barrier. Re-run the size sweep, pick the right activity counter, and give the barrier a data-size term. The rest of the loop is untouched.

A different kind of failure

A power cap, a memory fault, a flapping cable: each one enters the loop at a known place. A power cap sits between power and clock; a memory fault sits at occupancy; a bad cable sits outside the loop, at the data exchange. We add the entry point, not a new model.

Where the model stops

The simulator models everything up to throttling. A real cooling failure can end with machines shutting themselves down and someone walking to the room. That last step is deliberately outside the model, and we say so before anyone asks.

One heartbeat, five gauges, three stages. Calibration is not a one-off: it is the routine that keeps the simulator honest.